

Lógica Computacional — Resumo Teórico

Gonçalo Sousa

Atualizado: Semana 1 (Aulas 1–2)

Conteúdo

Aula 1 — Introdução à Lógica e Sintaxe da Lógica Proposicional	3
O que é a lógica e porque interessa à informática	3
Proposições	3
Conectivos lógicos	4
Linguagem da lógica proposicional	4
Semântica: valorações e tabelas de verdade	9
Satisfazibilidade, tautologias e contradições	10
 Aula 2 — Consequência Semântica, Equivalência e Conectivos Completos	 12
A relação de satisfação $\models_v$	12
Satisfazibilidade de um conjunto de fórmulas	13
Consequência semântica e equivalência	13
Propriedades da relação $\models$	14
Quantas proposições podem ser simultaneamente verdadeiras	16
Conjuntos completos de conectivos	17
Aplicação: modelar problemas em lógica proposicional	18
Ligação com a prática	20

Aula 1 — Introdução à Lógica e Sintaxe da Lógica Proposicional

O que é a lógica e porque interessa à informática

A **lógica** estuda os métodos e princípios que distinguem um raciocínio correto de um incorreto. A lógica formal nasce com Aristóteles (silogismos: regras de inferência como o *modus ponens* — de $A \rightarrow B$ e A infere-se B), mas só se torna **lógica matemática** a partir do séc. XIX (Boole, Frege, Hilbert, Gödel...), associada a problemas de fundamentação da matemática: teoria de modelos, teoria da dedução (prova) e sistemas axiomáticos.

Atenção

Cuidado com silogismos “parecidos” que não são válidos. “Todos os homens são mortais; Sócrates é homem; logo Sócrates é mortal” é válido. Mas “Todos os homens são mortais; Sócrates era mortal; logo todos os homens são Sócrates” **não é válido**, apesar de ter a mesma aparência superficial — a forma do argumento é que importa, não o facto de as frases “soarem” parecidas.

Uma **lógica** (no sentido abstrato) é composta por três partes:

1. uma **linguagem/sintaxe** — um alfabeto e regras para construir *fórmulas*;
2. uma **semântica** — uma noção do que é verdadeiro (pode haver várias interpretações semânticas para a mesma sintaxe);
3. um **sistema de dedução** — um conjunto finito de regras que permite obter fórmulas novas a partir de outras (raciocinar). Interessa que seja **correto** (*sound*): tudo o que se deduz é verdadeiro; e idealmente **completo**: tudo o que é verdadeiro é dedutível (nem sempre é possível ter as duas coisas — é isso que o curso vai explorar com os teoremas de correção e completude).

Neste curso trabalha-se sobretudo com a **lógica clássica** (proposicional e de primeira ordem). Há outras famílias de lógica — lógica de ordem superior, lógicas sub-estruturais (ex: intuicionista, onde $\neg\neg A \not\leftrightarrow A$, ao contrário do que veremos aqui) e lógicas modais (onde a verdade é parametrizada, ex: por tempo — “Hoje é segunda-feira” só é verdade nalguns “mundos”). É útil ter estas categorias em mente para perceber porque é que a lógica clássica que vamos estudar faz as suposições que faz (por exemplo, todas as proposições são verdadeiras ou falsas — nunca as duas nem nenhuma — o chamado *princípio do terceiro excluído*, que não vale em lógica intuicionista).

A lógica é relevante para a informática em várias frentes: circuitos digitais (lógica booleana), inteligência artificial, autómatos finitos (correspondem a fórmulas de lógica monádica de segunda ordem), XML/documentos estruturados, classes de complexidade caracterizadas por classes de fórmulas, linguagens de programação (programação lógica como Prolog; sistemas de tipos = sistemas dedutivos), verificação e especificação formal (hardware, software, protocolos de segurança), dedução automática (SAT/SMT solvers) e provadores de teoremas assistidos (Coq, Isabelle, Agda).

Proposições

Definição — Proposição

Uma **proposição** é uma frase declarativa à qual se pode atribuir um valor de verdade, **V** (verdadeiro) ou **F** (falso) — e apenas um dos dois. Frases que não afirmam nada com um valor de verdade definido (perguntas, ordens, expressões sem verbo, ou frases com variáveis livres cujo valor de verdade depende de algo não especificado) **não são proposições**.

Exemplo resolvido — Exercício 1.1 (lab, proplogic.pdf)

Para cada frase, indicar se é proposição e, sendo, o seu valor de verdade:

- (a) “O Porto é a capital de Portugal.” — É proposição. **F** (a capital é Lisboa).
- (b) “Em 1910 foi proclamada a República Portuguesa.” — É proposição. **V** (5 de outubro de 1910).
- (c) “Hei de comparar-te a um dia de verão?” — **Não é proposição**: é uma pergunta (interrogativa), não uma afirmação — não tem valor de verdade.
- (d) “ $1 + 2 + 3$ ” — **Não é proposição**: é uma expressão/termo (um número), não uma afirmação. Não diz nada que possa ser verdadeiro ou falso.
- (e) “ $1 + 2 + 3 = 6$ ” — É proposição. **V**.
- (f) “7 é um número par.” — É proposição. **F**.
- (g) “ $x + y$ é um número par.” — **Não é proposição**: x e y são *variáveis livres* — o valor de verdade depende de que valores lhes atribuímos (é verdade para $x = 1, y = 1$, falso para $x = 1, y = 2$). A frase só passa a ter um valor de verdade fixo depois de fixarmos x e y .
- (h) “Existe um x tal que $7 + x$ é um número par.” — É proposição, e **V**: apesar de ter uma variável, o quantificador “existe um x ” **fecha** a variável — a frase inteira já não depende de nenhum x externo (basta $x = 1$: $7 + 1 = 8$ é par). Compara com (g): a diferença entre uma variável livre (não é proposição) e uma variável quantificada (é proposição) só fica completamente formal em lógica de primeira ordem, mais à frente no curso — mas já vale a pena notar a distinção agora.

Conectivos lógicos

Cada proposição simples é representada por uma **variável proposicional** ($p, q, r, \dots$), e proposições compostas obtêm-se combinando-as com **conectivos**:

Conectivo	Símbolo	Aridade	Outros símbolos equivalentes
Conjunção (“e”)	$\wedge$	2	$\&, \&\&, \cdot$
Disjunção (“ou”)	$\vee$	2	$, +$
Negação (“não”)	$\neg$	1	$\sim, -, !$
Implicação (“se...então”)	$\rightarrow$	2	$\Rightarrow, \supset$

Linguagem da lógica proposicional**Definição — Alfabeto**

A linguagem da lógica proposicional é construída a partir dos símbolos primitivos:

- um conjunto **numerável** de variáveis proposicionais $\mathcal{V}_{Prop} = \{p, q, r, \dots, p_1, \dots\}$;
- os conectivos lógicos $\wedge, \vee, \neg, \rightarrow$;
- os parêntesis (e).

Definição — Fórmula bem formada

O conjunto das **fórmulas** ($\phi, \psi, \theta, \dots$) é definido indutivamente:

1. uma variável proposicional p é uma fórmula;
2. se ϕ é fórmula, então $(\neg\phi)$ é fórmula;
3. se ϕ e ψ são fórmulas, então $(\phi \wedge \psi)$, $(\phi \vee \psi)$ e $(\phi \rightarrow \psi)$ são fórmulas.

Nada mais é fórmula. Exemplo: $((p \wedge (\neg p)) \rightarrow (\neg(p \wedge (q \vee r))))$.

Esta definição indutiva é o que permite desenhar a **árvore sintática** de uma fórmula: a raiz é o conectivo principal, e os seus filhos são as fórmulas que ele combina (recursivamente, até chegar às variáveis, que são as folhas).

Convenção para omitir parêntesis

Escrever todos os parêntesis torna as fórmulas ilegíveis, por isso usa-se uma convenção de precedência (tal como em aritmética, onde $\times$ tem prioridade sobre $+$):

- os parêntesis mais exteriores podem ser omitidos;
- $\neg$ tem prioridade sobre $\wedge$;
- $\wedge$ tem prioridade sobre $\vee$;
- $\vee$ tem prioridade sobre $\rightarrow$;
- $\wedge$ e $\vee$ são **associativos à esquerda**;
- $\rightarrow$ é **associativo à direita**.

Exemplo resolvido

- $\phi \wedge \psi \vee \theta$ abrevia $((\phi \wedge \psi) \vee \theta)$ (o $\wedge$ “agarra” primeiro).
- $p \wedge \neg p \rightarrow \neg(p \wedge (q \vee r))$ corresponde a $((p \wedge (\neg p)) \rightarrow (\neg(p \wedge (q \vee r))))$.
- $p \rightarrow q \rightarrow r$ corresponde a $p \rightarrow (q \rightarrow r)$ (associatividade à direita do $\rightarrow$: lê-se “de dentro para fora”, começando pelo lado direito).

Definição — Subfórmula

Uma **subfórmula imediata** de ψ é: (1) nenhuma, se ψ é uma variável; (2) ϕ , se $\psi = \neg\phi$; (3) ϕ e ψ' , se $\psi = \phi \wedge \psi'$, $\phi \vee \psi'$ ou $\phi \rightarrow \psi'$.

ϕ é **subfórmula** de ψ se ϕ é subfórmula imediata de ψ , ou se existe θ tal que ϕ é subfórmula de θ e θ é subfórmula de ψ (fecho transitivo das subfórmulas imediatas). Por convenção, ψ é sempre subfórmula de si própria.

O exercício seguinte junta a convenção de precedência, a árvore sintática e a noção de subfórmula — a melhor forma de treinar os três em conjunto é desenhar a árvore completa e depois ler as subfórmulas diretamente dos nós dessa árvore (cada nó interno, incluindo a raiz, é uma subfórmula; as folhas são as variáveis).

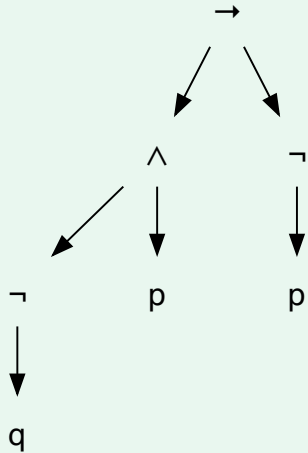
Exemplo resolvido — Exercício 1.3(a) (lab, proplogic.pdf)

Fórmula: $p \wedge \neg q \rightarrow \neg p$.

Passo 1 — reintroduzir os parêntesis pela convenção de precedência. $\neg$ agarra primeiro: $\neg q$ e $\neg p$. Depois $\wedge$: $p \wedge (\neg q)$. Por fim $\rightarrow$ (o conectivo de menor prioridade, por isso é a raiz):

$$(p \wedge (\neg q)) \rightarrow (\neg p)$$

Passo 2 — árvore sintática (raiz = conectivo principal, $\rightarrow$):

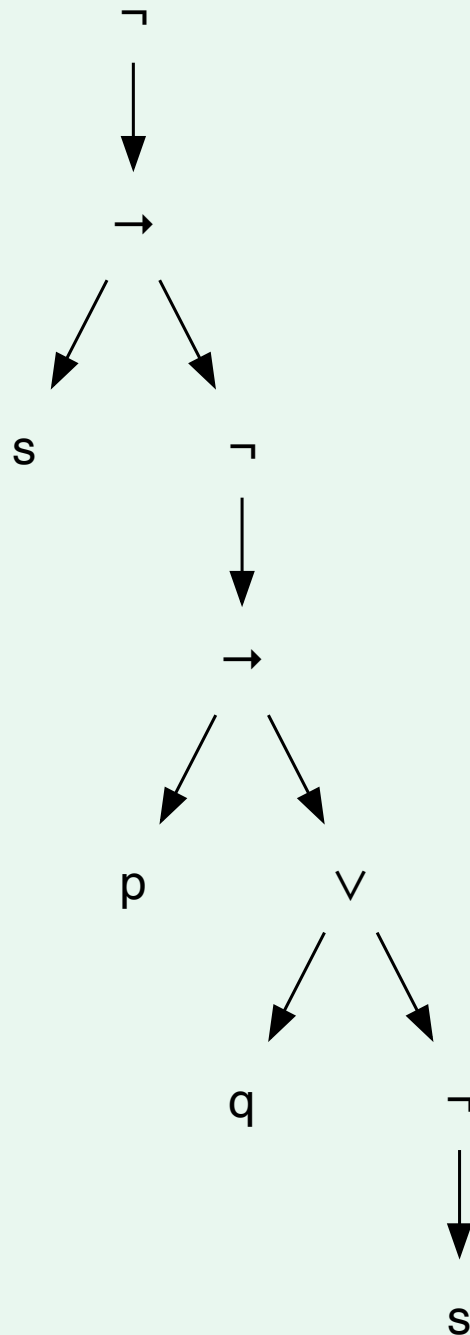


Passo 3 — lista de subfórmulas (lidas diretamente dos nós da árvore, da raiz para as folhas):
 $p \wedge \neg q \rightarrow \neg p$ (a própria fórmula), $p \wedge \neg q$, $\neg p$, p , $\neg q$, q .

Exemplo resolvido — Exercício 1.3(c) (lab, proplogic.pdf)**Fórmula:** $\neg(s \rightarrow (\neg(p \rightarrow (q \vee \neg s))))$.

Aqui os parêntesis já vêm quase todos explícitos no enunciado — só falta decompor “de fora para dentro” para identificar cada conectivo principal. Vamos dar um nome a cada subfórmula à medida que aparece, de dentro para fora (é a ordem inversa da leitura, mas é a ordem em que a árvore se constrói):

- $A = q \vee \neg s$ (disjunção de q com $\neg s$)
- $B = p \rightarrow A$
- $C = \neg B$
- $D = s \rightarrow C$
- $E = \neg D$ (a fórmula toda)

Árvore sintática:

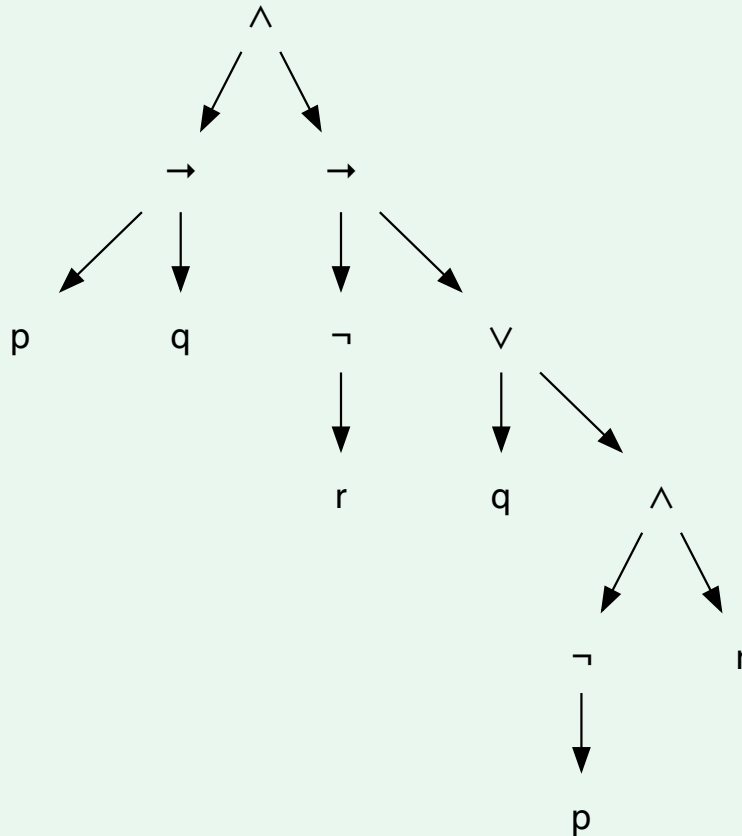
Exemplo resolvido — Exercício 1.3(e) (lab, proplogic.pdf)

Fórmula: $(p \rightarrow q) \wedge (\neg r \rightarrow (q \vee (\neg p \wedge r)))$.

De novo os parêntesis já resolvem a ambiguidade; só falta identificar a estrutura. O conectivo principal é o $\wedge$ mais exterior, que combina dois ramos:

- ramo esquerdo: $L = p \rightarrow q$
- ramo direito: $M = \neg r \rightarrow M_2$, onde $M_2 = q \vee M_1$ e $M_1 = \neg p \wedge r$

Árvore sintática:



Subfórmulas: a fórmula toda, $L = p \rightarrow q$, p , q , $M = \neg r \rightarrow (q \vee (\neg p \wedge r))$, $\neg r$, r , $M_2 = q \vee (\neg p \wedge r)$, q , $M_1 = \neg p \wedge r$, $\neg p$, p , r .

Atenção

Nota adicional (não estava explícito nos slides, mas é essencial para não errar nos exercícios): quando reconstróis os parêntesis pela convenção de precedência, faz sempre isso **antes** de tentar desenhar a árvore — tentar fazer as duas coisas ao mesmo tempo é a fonte mais comum de erros. Também repara que o conectivo **principal** de uma fórmula (a raiz da árvore) é sempre o de **menor prioridade** entre os que aparecem “soltos” (não dentro de parêntesis explícitos) — por isso em 1.3(a) a raiz é $\rightarrow$ e não $\wedge$, mesmo aparecendo $\wedge$ mais à esquerda na leitura da fórmula.

Semântica: valorações e tabelas de verdade

Definição — Valoração (atribuição de valores de verdade)

Uma **valoração** é uma função $v : \mathcal{V}_{Prop} \rightarrow \{V, F\}$ que atribui um valor de verdade a cada variável proposicional. Estende-se ao conjunto das fórmulas por indução na estrutura:

1. $v(p)$ já está definido, para $p \in \mathcal{V}_{Prop}$;
2. $v(\neg\phi) = V$ se $v(\phi) = F$, e $v(\neg\phi) = F$ se $v(\phi) = V$;
3. $v(\phi \wedge \psi) = V$ sse $v(\phi) = V$ e $v(\psi) = V$ (senão F);
4. $v(\phi \vee \psi) = V$ sse $v(\phi) = V$ ou $v(\psi) = V$ (senão F);
5. $v(\phi \rightarrow \psi) = F$ sse $v(\phi) = V$ e $v(\psi) = F$ (senão V).

Atenção

O caso mais contra-intuitivo é a implicação: $\phi \rightarrow \psi$ só é **falsa** quando o antecedente é verdadeiro e o consequente é falso. Em particular, sempre que o antecedente ϕ é **falso**, a implicação inteira é **verdadeira**, independentemente do consequente (“de uma premissa falsa pode concluir-se o que quiser” — *ex falso quodlibet*). Isto vai ser usado constantemente nos exercícios de classificação de fórmulas (ver Exercício 1.9 mais abaixo) e nas provas “sem construir a tabela de verdade” (Exercício 1.14).

ϕ	$\neg\phi$	ϕ	ψ	$\phi \wedge \psi$	ϕ	ψ	$\phi \vee \psi$	ϕ	ψ	$\phi \rightarrow \psi$
F	V	F	F	F	F	F	F	F	F	V
V	F	F	V	F	F	V	V	F	V	V
		V	F	F	V	F	V	V	F	F
		V	V	V	V	V	V	V	V	V

Uma fórmula ϕ com n variáveis proposicionais tem 2^n valorações possíveis (cada variável pode ser V ou F , independentemente das outras) — constrói-se uma **tabela de verdade** com uma linha por valoração, calculando o valor de ϕ coluna a coluna (dos operadores mais “internos” para o mais “externo”, exatamente como na árvore sintática).

Exemplo resolvido — Exercício 1.4(b) (lab, proplogic.pdf)

Construir a tabela de verdade de $\neg q \wedge (p \rightarrow q)$ (2 variáveis, logo 4 linhas):

p	q	$\neg q$	$p \rightarrow q$	$\neg q \wedge (p \rightarrow q)$
V	V	F	V	F
V	F	V	F	F
F	V	F	V	F
F	F	V	V	V

A fórmula só é verdadeira quando p e q são ambos falsos. Isto não é coincidência: por equivalências semânticas (ver mais abaixo), $\neg q \wedge (p \rightarrow q) \Leftrightarrow \neg q \wedge (\neg p \vee q) \Leftrightarrow (\neg q \wedge \neg p) \vee (\neg q \wedge q) \Leftrightarrow (\neg p \wedge \neg q) \vee F \Leftrightarrow \neg p \wedge \neg q$ — exatamente o que a tabela mostra.

Exemplo resolvido — Exercício 1.4(c) (lab, proplogic.pdf)

Construir a tabela de verdade de $(p \wedge q) \rightarrow \neg(p \vee q)$:

p	q	$p \wedge q$	$p \vee q$	$\neg(p \vee q)$	$(p \wedge q) \rightarrow \neg(p \vee q)$
V	V	V	V	F	F
V	F	F	V	F	V
F	V	F	V	F	V
F	F	F	F	V	V

Só é falsa na linha $p = V, q = V$: aí o antecedente $p \wedge q$ é verdadeiro mas o consequente $\neg(p \vee q)$ é falso (porque $p \vee q$ é verdadeiro). Em todas as outras linhas o antecedente $p \wedge q$ já é falso, o que torna a implicação automaticamente verdadeira (ver nota acima sobre a implicação).

Satisfazibilidade, tautologias e contradições**Definição — Satisfazibilidade, tautologia, contradição**

Uma fórmula ϕ é:

- **satisfazível** se existe uma valoração v tal que $v(\phi) = V$ — escreve-se $\models_v \phi$ e diz-se que v **satisfaz** ϕ ;
- **tautologia** se **todas** as valorações v dão $v(\phi) = V$ — escreve-se $\models \phi$ (ex: $\models p \vee \neg p$, o princípio do terceiro excluído);
- **contradição** se **todas** as valorações v dão $v(\phi) = F$ — escreve-se $\not\models \phi$ (ex: $\not\models p \wedge \neg p$).

Uma fórmula é **insatisfazível** sse é uma contradição. Uma fórmula satisfazível mas não tautologia diz-se por vezes **contingente**: é verdadeira nalgumas valorações e falsa noutras.

Exemplo resolvido — Exercício 1.9 (lab, proplogic.pdf) — os três casos

Classificar as seguintes fórmulas como tautologia, satisfazível (mas não tautologia) ou contradição, justificando sem tabela completa quando possível.

(d) $p \rightarrow (q \rightarrow (p \vee q))$ — é tautologia. Se $v(p) = V$: como $p \vee q$ já é V (porque p é V), o consequente $q \rightarrow (p \vee q)$ é $V \rightarrow V = V$ qualquer que seja q ; logo a implicação exterior é $V \rightarrow V = V$. Se $v(p) = F$: a implicação exterior tem antecedente falso, logo é automaticamente V . Em ambos os casos dá V , para qualquer q — é tautologia.

(i) $(p \vee q) \rightarrow (p \wedge q)$ — é satisfazível, mas não tautologia nem contradição. Com $v(p) = V, v(q) = V$: antecedente V , consequente V , logo $V \rightarrow V = V$ (satisfazível — existe valoração que a torna verdadeira). Mas com $v(p) = V, v(q) = F$: antecedente $p \vee q = V$, consequente $p \wedge q = F$, logo $V \rightarrow F = F$ (existe também valoração que a torna falsa). Como há valorações dos dois tipos, não é tautologia nem contradição.

(j) $(p \vee \neg q) \wedge (\neg p \wedge q)$ — é contradição. O segundo conjunto, $\neg p \wedge q$, só é verdadeiro quando $p = F$ e $q = V$. Nesse caso, o primeiro conjunto, $p \vee \neg q$, calcula-se com $p = F$ e $q = V$: $F \vee \neg V = F \vee F = F$. Ou seja, exatamente quando o segundo conjunto poderia ser verdadeiro, o primeiro é forçosamente falso — logo a conjunção inteira nunca é verdadeira, para nenhuma valoração: é contradição.

Atenção

Nota adicional: repara na técnica usada em (d), (i) e (j) — em vez de construir sempre a tabela de verdade completa (4 linhas, no caso de 2 variáveis), tentámos primeiro **fixar o valor de uma subfórmula-chave** (o antecedente da implicação, ou um dos conjuntos) e ver o que isso força no resto da fórmula. É mais rápido e, mais importante, é a técnica que vais precisar no Exercício 1.14 (“sem construir a tabela de verdade”), onde 2 ou mais variáveis tornam a tabela completa demasiado grande para ser prático.

Aula 2 — Consequência Semântica, Equivalência e Conectivos Completos

A relação de satisfação $\models_v$

A extensão de v às fórmulas (aula anterior) pode ser reescrita como uma relação $\models_v$, definida indutivamente na estrutura da fórmula — é apenas outra notação para a mesma coisa, mas é a notação usada daqui em diante:

Definição — Relação de satisfação

Dada uma valoração v , a relação $\models_v$ define-se por:

1. $\models_v p$ sse $v(p) = V$;
2. $\models_v \neg\phi$ sse $\not\models_v \phi$;
3. $\models_v \phi \wedge \psi$ sse $\models_v \phi$ e $\models_v \psi$;
4. $\models_v \phi \vee \psi$ sse $\models_v \phi$ ou $\models_v \psi$;
5. $\models_v \phi \rightarrow \psi$ sse $\not\models_v \phi$ ou $\models_v \psi$.

Diz-se que v **satisfaz** ϕ se $\models_v \phi$.

Exemplo resolvido

Com $v(p) = V, v(q) = v(r) = F$, para saber se $\models_v (p \rightarrow (q \vee r)) \vee (r \rightarrow \neg p)$ basta que **um** dos dois lados da disjunção seja satisfeito. O lado direito, $r \rightarrow \neg p$, é satisfeito porque $\not\models_v r$ (implicação com antecedente falso). Logo $\models_v (p \rightarrow (q \vee r)) \vee (r \rightarrow \neg p)$ — nem foi preciso avaliar o lado esquerdo.

Exemplo resolvido — Exercício 1.5(b) (lab, proplogic.pdf)

Com $v(p) = V$ e $v(q) = F$, verificar se $\models_v (p \vee \neg q) \wedge (\neg p \wedge q)$.

O segundo conjunto, $\neg p \wedge q$: temos $v(p) = V$ logo $\not\models_v \neg p$ (já falha aqui). Como um conjunto ($\wedge$) precisa dos **dois** lados satisfeitos e um deles já falha, $\not\models_v \neg p \wedge q$. Logo, mesmo sem avaliar o primeiro conjunto, a conjunção inteira falha: $\not\models_v (p \vee \neg q) \wedge (\neg p \wedge q)$.

Exemplo resolvido — Exercício 1.5(e) (lab, proplogic.pdf)

Com $v(p) = V$ e $v(q) = F$, verificar se $\models_v ((p \rightarrow q) \rightarrow p) \rightarrow p$.

$p \rightarrow q$: $v(p) = V, v(q) = F$, logo $\not\models_v p \rightarrow q$ (antecedente verdadeiro, consequente falso — o único caso em que a implicação falha). $(p \rightarrow q) \rightarrow p$: antecedente falso ($\not\models_v p \rightarrow q$), logo esta implicação é automaticamente satisfeita: $\models_v (p \rightarrow q) \rightarrow p$. Fórmula toda: $((p \rightarrow q) \rightarrow p) \rightarrow p$: antecedente satisfeito ($\models_v (p \rightarrow q) \rightarrow p$) e consequente satisfeito ($\models_v p$, porque $v(p) = V$) — logo $\models_v ((p \rightarrow q) \rightarrow p) \rightarrow p$.

Frequente em exame

$((p \rightarrow q) \rightarrow p) \rightarrow p$ é a **Lei de Peirce**, e é de facto uma **tautologia** (é verdadeira para *qualquer* v , não só para o v deste exercício — podes confirmar construindo a tabela completa com as 4 valorações de p, q). É um resultado clássico e frequentemente pedido: é um dos exemplos-padrão de tautologia que **não é válida em lógica intuicionista** (relembra a nota da Aula 1 sobre lógicas sub-estruturais) — distingue-a da lógica clássica que estamos a estudar.

Atenção

Nota adicional — armadilha comum: **não confundir** $(p \rightarrow q) \rightarrow p$ (que aparece na Lei de Peirce acima) com $(q \rightarrow p) \rightarrow p$ (parecida, mas com p e q trocados no antecedente). Ao contrário da primeira, $(q \rightarrow p) \rightarrow p$ **não** é tautologia — falha, por exemplo, com $v(p) = F$: nesse caso $q \rightarrow p = \neg q$ (verdadeiro só se q for falso), e se $v(q) = F$ então $q \rightarrow p$ é verdadeiro, tornando $(q \rightarrow p) \rightarrow p$ igual a $V \rightarrow F = F$. A posição exata de cada variável na fórmula importa — nunca assumas que uma fórmula “parecida” com uma tautologia conhecida também o é.

Satisfazibilidade de um conjunto de fórmulas**Definição — Satisfazibilidade de um conjunto**

Seja Γ um conjunto de fórmulas. Uma valoração v **satisfaz** Γ (notação $\models_v \Gamma$) se satisfaz **todas** as fórmulas de Γ : $\forall \psi \in \Gamma, \models_v \psi$. Γ é **satisfazível** se existe alguma valoração que o satisfaz.

Consequência semântica e equivalência**Definição — Consequência semântica ($\models$)**

ϕ é **consequência semântica** de Γ (notação $\Gamma \models \phi$) se **toda** valoração v que satisfaz Γ também satisfaz ϕ .

Casos especiais de notação:

- $\emptyset \models \phi$ escreve-se $\models \phi$ — equivale a ϕ ser tautologia (nenhuma condição sobre v : tem de valer sempre).
- Se $\Gamma = \{\psi\}$, escreve-se $\psi \models \phi$.
- Se $\psi \models \phi$ e $\phi \models \psi$, ψ e ϕ dizem-se **semanticamente equivalentes**, escreve-se $\psi \Leftrightarrow \phi$ (têm exatamente o mesmo valor de verdade em qualquer valoração).

Definição — Leis de equivalência semântica

Lei	Equivalência
Comutatividade	$\phi \wedge \psi \Leftrightarrow \psi \wedge \phi$; $\phi \vee \psi \Leftrightarrow \psi \vee \phi$
Lei de De Morgan	$\neg(\phi \wedge \psi) \Leftrightarrow \neg\phi \vee \neg\psi$; $\neg(\phi \vee \psi) \Leftrightarrow \neg\phi \wedge \neg\psi$
Associatividade	$(\phi \wedge \psi) \wedge \theta \Leftrightarrow \phi \wedge (\psi \wedge \theta)$; idem para $\vee$
Idempotência	$\phi \vee \phi \Leftrightarrow \phi$; $\phi \wedge \phi \Leftrightarrow \phi$
Distributividade	$(\phi \wedge \psi) \vee \theta \Leftrightarrow (\phi \vee \theta) \wedge (\psi \vee \theta)$; $(\phi \vee \psi) \wedge \theta \Leftrightarrow (\phi \wedge \theta) \vee (\psi \wedge \theta)$
Dupla negação	$\neg\neg\phi \Leftrightarrow \phi$
Implicação	$\phi \rightarrow \psi \Leftrightarrow \neg\phi \vee \psi$

Estas leis funcionam como uma “álgebra booleana”: permitem transformar uma fórmula noutra equivalente sem ter de recorrer à tabela de verdade — substituindo subfórmulas pelo seu equivalente, passo a passo (tal como se faz álgebra normal, mas com $\wedge, \vee, \neg$ em vez de $\times, +$).

Exemplo resolvido — Exercício 1.6(a) (lab, proplogic.pdf)

Mostrar que $\neg(p \rightarrow q) \Leftrightarrow p \wedge \neg q$, usando as leis (sem tabela de verdade):

$$\neg(p \rightarrow q) \stackrel{\text{(implicação)}}{\Leftrightarrow} \neg(\neg p \vee q) \stackrel{\text{(De Morgan)}}{\Leftrightarrow} \neg\neg p \wedge \neg q \stackrel{\text{(dupla negação)}}{\Leftrightarrow} p \wedge \neg q$$

Cada passo substitui uma subfórmula pelo seu equivalente segundo uma única lei da tabela — é assim que se lê/escreve qualquer prova por equivalências.

Exemplo resolvido — Exercício 1.6(e) (lab, proplogic.pdf)

Mostrar que $(p \vee q) \rightarrow r \Leftrightarrow (p \rightarrow r) \wedge (q \rightarrow r)$:

$$(p \vee q) \rightarrow r \stackrel{\text{(implicação)}}{\Leftrightarrow} \neg(p \vee q) \vee r \stackrel{\text{(De Morgan)}}{\Leftrightarrow} (\neg p \wedge \neg q) \vee r$$

Agora aplica-se a distributividade — $(\phi \wedge \psi) \vee \theta \Leftrightarrow (\phi \vee \theta) \wedge (\psi \vee \theta)$ — com $\phi = \neg p$, $\psi = \neg q$, $\theta = r$:

$$(\neg p \wedge \neg q) \vee r \stackrel{\text{(distributividade)}}{\Leftrightarrow} (\neg p \vee r) \wedge (\neg q \vee r) \stackrel{\text{(implicação, 2x)}}{\Leftrightarrow} (p \rightarrow r) \wedge (q \rightarrow r)$$

Atenção

Guarda a equivalência do Exercício 1.6(e) — $(p \vee q) \rightarrow r \Leftrightarrow (p \rightarrow r) \wedge (q \rightarrow r)$ — porque vai reaparecer, já pronta a reutilizar, no Exercício 1.13(d) mais abaixo (justificação de consequência semântica sem trabalho extra).

Exemplo resolvido — Exercício 1.8(a) e 1.8(b) (lab, proplogic.pdf) — distribuir a implicação sobre a conjunção

(a) $(p \rightarrow q) \wedge (p \rightarrow r) \Leftrightarrow p \rightarrow (q \wedge r)$ — **é válida**. Prova pelas leis:

$$(p \rightarrow q) \wedge (p \rightarrow r) \Leftrightarrow (\neg p \vee q) \wedge (\neg p \vee r) \stackrel{\text{(distributividade)}}{\Leftrightarrow} \neg p \vee (q \wedge r) \Leftrightarrow p \rightarrow (q \wedge r)$$

(b) $(p \rightarrow r) \wedge (q \rightarrow r) \Leftrightarrow (p \wedge q) \rightarrow r$ — **NÃO é válida**. Contraexemplo: $v(p) = V, v(q) = F, v(r) = F$. Lado esquerdo: $p \rightarrow r = V \rightarrow F = F$; logo $(p \rightarrow r) \wedge (q \rightarrow r) = F$. Lado direito: $p \wedge q = V \wedge F = F$; logo $(p \wedge q) \rightarrow r = F \rightarrow F = V$. Lado esquerdo é F , lado direito é V — **não são equivalentes**.

Atenção

Compara com atenção (a) e (b): olham parecidas (ambas “distribuem $\rightarrow$ sobre $\wedge$ ”), mas só a que distribui $\wedge$ do lado do **consequente** (a) é válida — distribuir sobre o **antecedente** (b) já não é. É um erro muito fácil de cometer sem verificar — usa sempre as leis ou um contraexemplo para confirmar, nunca “por analogia visual”.

Propriedades da relação $\models$

As afirmações seguintes envolvem $\Gamma \models \theta$ de forma mais abstrata (sem fixar fórmulas concretas) — testam se percebeste bem a **definição** de consequência semântica, não cálculo. A técnica geral: para mostrar que uma afirmação é **verdadeira**, usa diretamente a definição; para mostrar que é **falsa**, basta um contraexemplo concreto (escolher Γ, Σ, θ específicos).

Exemplo resolvido — Exercício 1.16(a), (b) e (f) (lab, proplogic.pdf) — monotonicidade

(a) Se $\Gamma \models \theta$ e $\Gamma \subseteq \Sigma$, então $\Sigma \models \theta$ — **VERDADEIRO**. Seja v uma valoração que satisfaz Σ . Como $\Gamma \subseteq \Sigma$, v satisfaz em particular todas as fórmulas de Γ (são também fórmulas de Σ), logo $\models_v \Gamma$. Por hipótese $\Gamma \models \theta$, logo $\models_v \theta$. Como isto vale para qualquer v que satisfaça Σ , conclui-se $\Sigma \models \theta$. (Intuição: **acrescentar premissas nunca destrói** uma consequência já válida — chama-se **monotonicidade**.)

(b) Se $\Sigma \models \theta$ e $\Gamma \subseteq \Sigma$, então $\Gamma \models \theta$ — **FALSO** (é o recíproco de (a), e a monotonicidade só funciona num sentido). Contraexemplo: $\theta = q$, $\Sigma = \{p, p \rightarrow q\}$, $\Gamma = \{p\} \subseteq \Sigma$. Tem-se $\Sigma \models q$ (modus ponens: qualquer v que satisfaça p e $p \rightarrow q$ satisfaz q). Mas $\Gamma \not\models q$: a valoração $v(p) = V, v(q) = F$ satisfaz $\Gamma = \{p\}$ mas não satisfaz q . **Menos premissas podem já não bastar** para a mesma conclusão.

(f) Se $\Sigma \models \theta$ e $\Sigma \subseteq \Gamma$, então $\Sigma \cup \Gamma \models \theta$ — **VERDADEIRO, e trivialmente**. Repara que, se $\Sigma \subseteq \Gamma$, então $\Sigma \cup \Gamma = \Gamma$ (a união de um conjunto com um dos seus sobreconjuntos é o sobreconjunto). A afirmação reduz-se exatamente a “ $\Sigma \models \theta$ e $\Sigma \subseteq \Gamma$, logo $\Gamma \models \theta$ ” — que é **o mesmo enunciado da alínea (a)**, com Γ e Σ trocados de nome. Não é preciso prova nova.

Atenção

A lição de (a)/(b)/(f): $\Gamma \models \theta$ é **monótono ao acrescentar premissas** (mais hipóteses, no máximo, só ajudam a concluir mais coisas — nunca deixam de valer uma consequência já estabelecida), mas **não é monótono ao remover premissas** — retirar uma hipótese pode fazer uma conclusão válida deixar de o ser. Isto explica intuitivamente porque é que (a) e (f) são verdadeiros (só acrescentam) e (b) é falso (remove).

Exemplo resolvido — Exercício 1.13 (lab, proplogic.pdf)

(a) $\models (p \wedge \neg p) \rightarrow (p \wedge \neg r)$ — **VERDADEIRO**. O antecedente $p \wedge \neg p$ é sempre falso (contradição), logo a implicação é sempre verdadeira, qualquer que seja o consequente — é tautologia.

(b) $\models (q \vee \neg r) \rightarrow (q \wedge \neg q)$ — **FALSO**. O consequente $q \wedge \neg q$ é sempre falso (contradição). Logo a implicação só é verdadeira quando o antecedente também é falso. Mas o antecedente $q \vee \neg r$ **não** é sempre falso: com $v(q) = V, v(r) = V$, $q \vee \neg r = V \vee F = V$, e então a implicação dá $V \rightarrow F = F$. Não é tautologia (contraexemplo: $q = V, r = V$).

(c) $\{(p \rightarrow q) \vee r, ((p \rightarrow q) \vee r) \rightarrow \neg r, (p \rightarrow q) \rightarrow (q \rightarrow r)\} \models \neg q$ — **VERDADEIRO**. Seja v uma valoração que satisfaz as três premissas. Chamando $\varphi_1 = (p \rightarrow q) \vee r$: como $\models_v \varphi_1$ e $\models_v \varphi_1 \rightarrow \neg r$, por modus ponens $\models_v \neg r$, ou seja $v(r) = F$. Com $r = F$, $\varphi_1 = (p \rightarrow q) \vee F$, e como $\models_v \varphi_1$, tem de ser $\models_v p \rightarrow q$. Agora a terceira premissa, $(p \rightarrow q) \rightarrow (q \rightarrow r)$: como $\models_v p \rightarrow q$, por modus ponens $\models_v q \rightarrow r$, isto é $q \rightarrow F$, que é equivalente a $\neg q$. Logo $\models_v \neg q$ — exatamente o que queríamos mostrar, para qualquer v que satisfaça as três premissas.

(d) $\{(p \vee q) \rightarrow r\} \models (p \rightarrow r) \vee (q \rightarrow r)$ — **VERDADEIRO**. Já sabemos do Exercício 1.6(e) que $(p \vee q) \rightarrow r \Leftrightarrow (p \rightarrow r) \wedge (q \rightarrow r)$. Ora $(p \rightarrow r) \wedge (q \rightarrow r)$ implica trivialmente $(p \rightarrow r) \vee (q \rightarrow r)$ (uma conjunção implica sempre a disjunção dos mesmos termos: se ambos são verdadeiros, em particular um deles é). Logo $(p \vee q) \rightarrow r \models (p \rightarrow r) \vee (q \rightarrow r)$.

Exemplo resolvido — Exercício 1.14(a) e 1.14(d) (lab, proplogic.pdf) — sem tabela de verdade

(a) $\models p \rightarrow (((q \rightarrow p) \rightarrow p) \rightarrow r)$ — **FALSO**. Analisa-se por casos em p . Se $v(p) = V$: então $q \rightarrow p$ é sempre V (consequente verdadeiro), logo $(q \rightarrow p) \rightarrow p = V \rightarrow V = V$, e a subfórmula $((q \rightarrow p) \rightarrow p) \rightarrow r$ reduz-se a $V \rightarrow r$, ou seja, ao próprio valor de r . Logo, com $p = V$, a fórmula inteira vale exatamente $v(r)$. Escolhendo $v(r) = F$ (e $v(q)$ arbitrário, por exemplo V), a fórmula dá F — não é tautologia.

Contraexemplo: $p = V, q = V, r = F$.

(d) $\models ((p \wedge q) \rightarrow (s \vee t)) \rightarrow ((p \rightarrow s) \vee (q \rightarrow t))$ — **VERDADEIRO**. Em vez de percorrer as 16 linhas da tabela (4 variáveis), procura-se diretamente uma forma do consequente ser falso. O consequente $(p \rightarrow s) \vee (q \rightarrow t)$ só é falso se **ambos** os lados forem falsos: $p \rightarrow s$ falso exige $p = V, s = F$; $q \rightarrow t$ falso exige $q = V, t = F$. Ou seja, o **único** caso em que o consequente poderia falhar é $p = V, q = V, s = F, t = F$. Mas nesse caso o antecedente $(p \wedge q) \rightarrow (s \vee t)$ vale $(V \wedge V) \rightarrow (F \vee F) = V \rightarrow F = F$ — ou seja, precisamente nesse caso o antecedente também é falso, o que torna a implicação exterior verdadeira ($F \rightarrow \text{algo} = V$). Como não há mais nenhuma forma de o consequente ser falso, a implicação exterior nunca é falsa: é tautologia.

Atenção

Nota adicional sobre a técnica “sem tabela de verdade”: o método geral é **tentar construir um contraexemplo** (fixar a fórmula toda a F : para uma implicação, isso força antecedente V e consequente F ; segue as consequências obrigatórias passo a passo). Se o processo chega a uma contradição (como em 1.14(d), onde a única forma de falhar o consequente também falha o antecedente), a fórmula é tautologia. Se o processo chega a uma atribuição consistente (como em 1.14(a)), encontre mesmo um contraexemplo. É a mesma ideia usada em 1.9(d)/(i)/(j) e em 1.13, só que aqui aplicada sistematicamente à procura do contraexemplo.

Quantas proposições podem ser simultaneamente verdadeiras**Exemplo resolvido — Exercício 1.7 (lab, proplogic.pdf)**

Atribuindo valores de verdade a p, q, r , quantas das cinco disjunções $p \vee \neg q, \neg p \vee q, q \vee r, q \vee \neg r, \neg q \vee \neg r$ podem ser simultaneamente verdadeiras?

Como há só 3 variáveis, é viável construir a tabela completa (8 linhas) e contar, em cada linha, quantas das 5 fórmulas são V :

p	q	r	$p \vee \neg q$	$\neg p \vee q$	$q \vee r$	$q \vee \neg r$	$\neg q \vee \neg r$	nº de V
V	V	V	V	V	V	V	F	4
V	V	F	V	V	V	V	V	5
V	F	V	V	F	V	F	V	3
V	F	F	V	F	F	V	V	3
F	V	V	F	V	V	V	F	3
F	V	F	F	V	V	V	V	4
F	F	V	V	V	V	F	V	4
F	F	F	V	V	F	V	V	4

O **máximo** é **5** (todas simultaneamente verdadeiras), atingido em $p = V, q = V, r = F$ — por exemplo, verifica-se que $\neg q \vee \neg r$ dá $F \vee V = V$ porque $r = F$. O **mínimo** ao longo de todas as valorações é **3** (nunca é possível ter menos de 3 verdadeiras ao mesmo tempo, e isso acontece em três linhas diferentes da tabela).

Conjuntos completos de conectivos

Definição — Conjunto completo de conectivos

Um conjunto de conectivos C é **completo** se, para qualquer função de verdade $f : \{V, F\}^n \rightarrow \{V, F\}$, existe uma fórmula ϕ com n variáveis, usando só conectivos de C , tal que ϕ **induz** exatamente f (isto é, $F_\phi = f$, onde $F_\phi(x_1, \dots, x_n) = v_\chi(\phi)$ para $v_\chi(p_i) = x_i$). Por outras palavras: C chega para exprimir *qualquer* função booleana possível.

Prova de que $\{\wedge, \vee, \neg\}$ é completo (dos slides, por indução em n , o número de variáveis):

- **Base ($n = 1$):** há exatamente 4 funções de verdade de aridade 1 (uma tabela com 2 linhas tem $2^2 = 4$ combinações possíveis de saída). Para cada uma, exhibe-se uma fórmula que a realiza: $f_1 = \text{“sempre F”} \rightarrow \phi_1 = p \wedge \neg p$; $f_2 = \neg p \rightarrow \phi_2 = \neg p$; $f_3 = p \rightarrow \phi_3 = p$; $f_4 = \text{“sempre V”} \rightarrow \phi_4 = p \vee \neg p$.
- **Passo indutivo:** assumindo que a propriedade vale para n , seja $f : \{V, F\}^{n+1} \rightarrow \{V, F\}$. Define-se $f_1(x_1, \dots, x_n) = f(x_1, \dots, x_n, V)$ e $f_2(x_1, \dots, x_n) = f(x_1, \dots, x_n, F)$ (fixando a última variável). Por hipótese de indução existem ϕ_1, ϕ_2 com $F_{\phi_i} = f_i$. Toma-se $\phi = (p_{n+1} \wedge \phi_1) \vee (\neg p_{n+1} \wedge \phi_2)$: quando $x_{n+1} = V$, ϕ reduz-se a ϕ_1 ; quando $x_{n+1} = F$, reduz-se a ϕ_2 — exatamente o comportamento de f .

Como consequência imediata (usando as leis de De Morgan e a lei da implicação para reescrever $\wedge$ e $\vee$), também $\{\neg, \rightarrow\}$ é completo: $\phi \wedge \psi \Leftrightarrow \neg(\phi \rightarrow \neg\psi)$ e $\phi \vee \psi \Leftrightarrow \neg\phi \rightarrow \psi$.

Exemplo resolvido — Exercício 1.19(a) (lab, proplogic.pdf) — negação, disjunção é completo

Mostrar que $p \rightarrow q$, $p \leftrightarrow q$ e $p \wedge q$ se escrevem só com $\neg$ e $\vee$ (a mesma técnica da prova acima, mas explicitada concretamente):

- $p \rightarrow q \Leftrightarrow \neg p \vee q$ (é diretamente a lei da implicação).
- $p \wedge q \Leftrightarrow \neg(\neg p \vee \neg q)$ (De Morgan aplicado ao contrário: $\neg\neg(p \wedge q) \Leftrightarrow \neg(\neg p \vee \neg q)$, e $\neg\neg(p \wedge q) \Leftrightarrow p \wedge q$).
- $p \leftrightarrow q \Leftrightarrow (p \rightarrow q) \wedge (q \rightarrow p)$. Usando os dois pontos anteriores para reescrever cada peça só com $\neg, \vee$: $p \rightarrow q \Leftrightarrow \neg p \vee q$ e $q \rightarrow p \Leftrightarrow \neg q \vee p$; e a conjunção dessas duas, pelo ponto anterior ($\phi \wedge \psi \Leftrightarrow \neg(\neg\phi \vee \neg\psi)$), fica:

$$p \leftrightarrow q \Leftrightarrow \neg(\neg(\neg p \vee q) \vee \neg(\neg q \vee p))$$

— uma fórmula só com $\neg$ e $\vee$, como pedido.

Exemplo resolvido — Exercício 1.19(b) (lab, proplogic.pdf) — NAND é universal

Define-se um novo conectivo $\bar{\wedge}$ (“NAND”) pela tabela: $V\bar{\wedge}V = F$; $V\bar{\wedge}F = F\bar{\wedge}V = F\bar{\wedge}F = V$ (ou seja, $p\bar{\wedge}q \Leftrightarrow \neg(p \wedge q)$: só é falso quando ambos são verdadeiros). Mostrar que $\neg p$, $p \wedge q$ e $p \vee q$ se escrevem só com $\bar{\wedge}$:

Negação: $p\bar{\wedge}p \Leftrightarrow \neg(p \wedge p) \Leftrightarrow \neg p$ (idempotência). Logo $\neg p = p\bar{\wedge}p$.

Conjunção: $p\bar{\wedge}q \Leftrightarrow \neg(p \wedge q)$, logo $\neg(p\bar{\wedge}q) \Leftrightarrow p \wedge q$. E $\neg X = X\bar{\wedge}X$ (pelo caso anterior, com $X = p\bar{\wedge}q$), logo:

$$p \wedge q = (p\bar{\wedge}q) \bar{\wedge} (p\bar{\wedge}q)$$

Disjunção: por De Morgan, $p \vee q \Leftrightarrow \neg(\neg p \wedge \neg q) \Leftrightarrow \neg p\bar{\wedge}\neg q$ (a própria definição de $\bar{\wedge}$ já é a negação de um $\wedge$). Substituindo $\neg p = p\bar{\wedge}p$ e $\neg q = q\bar{\wedge}q$:

$$p \vee q = (p\bar{\wedge}p) \bar{\wedge} (q\bar{\wedge}q)$$

Como $\{\neg, \wedge, \vee\}$ já é completo e todos os três se escrevem só com $\bar{\wedge}$, conclui-se que $\{\bar{\wedge}\}$ **sozinho já é um conjunto completo de conectivos** — é o princípio por trás dos circuitos NAND em eletrónica digital (qualquer circuito lógico pode ser construído só com portas NAND).

Aplicação: modelar problemas em lógica proposicional

Os problemas seguintes não introduzem teoria nova — mostram como usar tudo o que já foi visto (tradução para fórmulas, tabelas de verdade, satisfazibilidade de conjuntos) para resolver problemas de raciocínio “do mundo real”. A técnica chave nos problemas de cavaleiros/vilões é a **auto-referência**: se a representa “A é cavaleiro” (diz sempre a verdade) e S é a fórmula que traduz a frase que A disse, então tem de valer $a \leftrightarrow S$ — A é cavaleiro sse o que disse é verdade.

Exemplo resolvido — Exercício 1.10(a) (lab, proplogic.pdf) — ilha dos cavaleiros e vilões

A diz: “Pelo menos um de nós [A, B] é vilão.” O que são A e B?

Seja a = “A é cavaleiro”, b = “B é cavaleiro”. A frase de A traduz-se por $S = \neg a \vee \neg b$. A condição de auto-referência é $a \leftrightarrow S$, isto é $a \leftrightarrow (\neg a \vee \neg b)$. Testam-se as 4 valorações de (a, b) :

a	b	$S = \neg a \vee \neg b$	$a \leftrightarrow S$
V	V	F	F
V	F	V	V
F	V	V	F
F	F	V	F

Só a linha $a = V, b = F$ é consistente (só nessa a biconditional dá V; nas outras a suposição contradiz-se a si própria). **A é cavaleiro, B é vilão.**

Exemplo resolvido — Exercício 1.10(e) (lab, proplogic.pdf) — três pessoas

A, B, C. A diz: “Somos todos vilões.” B diz: “Exatamente um de nós é cavaleiro.” O que são A, B e C? Sejam a, b, c os indicadores “é cavaleiro” de A, B, C. Traduzindo: $S_A = \neg a \wedge \neg b \wedge \neg c$ (todos vilões) e $S_B = (b \wedge \neg c) \vee (\neg b \wedge c)$ (exatamente um entre B e C é cavaleiro — note-se que S_B já não depende de a diretamente, mas a condição “exatamente um dos três” só se decompõe assim depois de se saber o valor de a ; para manter a tabela simples, calcula-se S_B como “exatamente um de $\{a, b, c\}$ ” diretamente nas 8 linhas).

Constrói-se a tabela com as duas condições de auto-referência, $\text{cond}_1 = (a \leftrightarrow S_A)$ e $\text{cond}_2 = (b \leftrightarrow S_B)$, para as 8 valorações de (a, b, c) — só interessam as linhas onde **ambas** são V:

a	b	c	S_A	cond_1	S_B (exat. 1 de a,b,c)	cond_2
V	V	V	F	F	F	F
V	V	F	F	F	F	F
V	F	V	F	F	F	V
V	F	F	F	F	V	F
F	V	V	F	V	F	F
F	V	F	F	V	V	V
F	F	V	F	V	F	F
F	F	F	V	F	V	F

Só a linha $a = F, b = V, c = F$ tem $\text{cond}_1 = V$ e $\text{cond}_2 = V$ simultaneamente. **A é vilão, B é cavaleiro, C é vilão** — e é a única solução consistente com as duas afirmações.

Exemplo resolvido — Exercício 1.11 (lab, proplogic.pdf) — suspeitos de um crime

Três suspeitos, A, B, C, fazem declarações: A: “B é culpado, mas C é inocente.” B: “Se A é culpado, então C é culpado.” C: “Eu sou inocente, mas um dos outros dois é culpado.”

Sejam g_a, g_b, g_c = “A/B/C é culpado”. Traduzindo cada declaração:

$$S_A = g_b \wedge \neg g_c \quad S_B = g_a \rightarrow g_c \quad S_C = \neg g_c \wedge (g_a \vee g_b)$$

Constrói-se **uma única tabela de verdade** com as 8 combinações de (g_a, g_b, g_c) e as três colunas S_A, S_B, S_C — e todas as alíneas seguintes respondem-se lendo linhas/colunas desta mesma tabela, sem repetir cálculos:

g_a	g_b	g_c	$S_A = g_b \wedge \neg g_c$	$S_B = g_a \rightarrow g_c$	$S_C = \neg g_c \wedge (g_a \vee g_b)$
V	V	V	F	V	F
V	V	F	V	F	V
V	F	V	F	V	F
V	F	F	F	F	V
F	V	V	F	V	F
F	V	F	V	V	V
F	F	V	F	V	F
F	F	F	F	V	F

(a) **As três declarações são compatíveis?** Procura-se uma linha com $S_A = S_B = S_C = V$: só a linha $g_a = F, g_b = V, g_c = F$. Sim, são compatíveis, e de forma **única** (só B culpado).

(b) **Alguma declaração é consequência das outras duas?** $\{S_A, S_B\} \models S_C$? As linhas com $S_A = V$ e $S_B = V$ simultaneamente: só a linha (F, V, F) (a segunda linha tem $S_A = V$ mas $S_B = F$, não conta) — e nessa linha $S_C = V$ também. Logo **sim**, $\{S_A, S_B\} \models S_C$. Já $\{S_A, S_C\} \models S_B$? As linhas com $S_A = V$ e $S_C = V$: a linha (V, V, F) e a linha (F, V, F) . Na primeira, $S_B = F$ — contraexemplo! Logo **não**, $\{S_A, S_C\} \not\models S_B$. E $\{S_B, S_C\} \models S_A$? Só a linha (F, V, F) tem $S_B = S_C = V$ em simultâneo, e aí $S_A = V$. Logo **sim**. (Nota como a relação de consequência não é simétrica entre os três pares — mais uma vez, a monotonicidade não “anda para trás”.)

(c) **Assumindo que os três são inocentes, quem mentiu?** Linha $g_a = F, g_b = F, g_c = F$: $S_A = F$ (A mentiu), $S_B = V$ (B disse a verdade), $S_C = F$ (C mentiu). **A e C mentiram; B disse a verdade.**

(d) **Assumindo que os três disseram a verdade, quem é inocente/culpado?** Precisa de $S_A = S_B = S_C = V$ — é exatamente a linha de (a): $g_a = F, g_b = V, g_c = F$. **Só B é culpado; A e C são inocentes.**

(e) **Assumindo que o inocente disse a verdade e o culpado mentiu, quem é inocente/-culpado?** Esta condição traduz-se, para cada pessoa X , por $S_X \leftrightarrow \neg g_X$ (inocente $\Leftrightarrow$ disse verdade). Percorrendo a tabela e comparando S_A com $\neg g_a$, S_B com $\neg g_b$ e S_C com $\neg g_c$ em cada linha, só a linha $g_a = V, g_b = F, g_c = V$ satisfaz as três condições ao mesmo tempo ($S_A = F = \neg g_a$; $S_B = V = \neg g_b$; $S_C = F = \neg g_c$). **A e C são culpados; B é inocente.**

Atenção

Repara como (c), (d) e (e) dão **três respostas diferentes** ao mesmo enunciado — cada uma corresponde a uma hipótese diferente sobre quem diz a verdade. É um erro comum tentar decidir “quem é mesmo culpado” sem antes fixar qual das três hipóteses está a ser assumida; a tabela de verdade única (com as colunas S_A, S_B, S_C) é o que permite responder às três sem ambiguidade nem trabalho repetido.

Ligação com a prática

Este resumo cobre e resolve uma seleção dos exercícios de `Semana_1/proplogic.pdf` (exercícios 1.1, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 1.10, 1.11, 1.13, 1.14, 1.16 e 1.19), escolhidos para cobrir todas as técnicas já dadas nas Aulas 1–2 sem repetir o mesmo tipo de raciocínio duas vezes. Ficam por fazer, sozinho, **para consolidar**: 1.2 (tradução mecânica, não precisa de exemplo resolvido), as restantes alíneas de 1.3, 1.4, 1.5, 1.6, 1.8, 1.9, 1.10, 1.11 (mesma técnica das que já foram resolvidas acima — usa os exemplos como modelo), 1.12 (mesma técnica de modelação do Exercício 1.11, mas com regras de escolha de cadeiras em vez de culpados), 1.15 (mesma ideia do Exercício 1.16, com $\subseteq$ e $\Leftrightarrow$ em vez de $\subseteq$ e $\models$).

Os exercícios **1.17, 1.18 (formas normais DNF/CNF)** e **1.20–1.23 (Horn, algoritmo de David-Putnam/DPLL)** ficam de fora deste resumo **de propósito** — ainda não foram dados nas teóricas (o programa da disciplina só chega a formas normais e algoritmos de satisfazibilidade mais à frente); entram no resumo quando a Aula correspondente for processada, para não estudar matéria fora de ordem.